# Introduction to the R package PFIM

Lucie Fayette [1]

[1]Université Paris Cité and Université Sorbonne Paris Nord, Inserm, IAME, F-75018 Paris, France

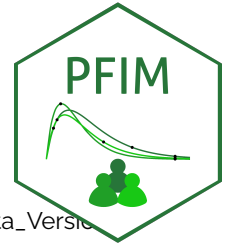
CESU Advanced course in population approaches in pharmacokinetics/pharmacodynamics

11/06/2026



# Introduction to the R package PFIM 7

- **PFIM** is a R package developed for FIM-based design evaluation and/or optimisation in Non-Linear Mixed-Effects Models (NLMEM)




- Version 7.0 released on CRAN in July 2025: https://github.com/packagePFIM/PFIM/tree/main/PFIM_7.1_beta_Versi
- Beta version 7.1 available on GitHub: https://github.com/packagePFIM/PFIM
- Evaluation and optimisation examples on GitHub

- Dedicated to pharmacometrics (PK/PD models)
- Based on Fisher Information Matrix
- Extension of earlier PFIM software versions

CRAN page of version 7.0 here!

Beta version 7.1 on GitHub here!

## Install beta version

1. Download the beta version https://github.com/packagePFIM/PFIM/tree/main/PFIM_7.1_beta_Version
2. Extract the contents of the archive to a specified location: myPath
3. In R, ensure that the devtools package is installed and load PFIM using

```
devtools::load_all(myPath)
```

## Workflow in PFIM7

1. Define model (PK/PD equations)
2. Specify parameter values
3. Define initial design
4. Compute FIM
5. Optimize design

**Output:**

- D-criterion
- Predicted standard errors (RSE)
- Design plot and sensitivy plots
- Power of covariate tests and number of subjects needed to reach target power
- Optimal sampling schedule

# Define model

```
# 1. MODEL DEFINITION
modelEquations = list(  "RespPK" = "dose_RespPK/V * exp(- Cl/V * t_RespPK) ",
                        "RespPD" = "E0 +  Emax * RespPK/( RespPK + C50 )" )
```

```
# Example with ODE
modelEquationsPKPD = list(
        "Deriv_RespPK" = "dose_RespPK/V * ka  * exp( -ka * t ) - Cl/V * RespPK",
        "Deriv_RespPD" = "Rin-kout*(1-Imax*RespPK/(RespPK+C50))*RespPD")
```

```
# Example with infusion
modelEquations = list( duringInfusion = list( "RespPK" = "dose_RespPK/Tinf_RespPK/Cl *(1 -
↪  exp(-Cl/V * t ) )" ),
                       afterInfusion  = list( "RespPK" = "(1 - exp(-Cl/V * t ) )" ) )
```

# Define parameters

```
# 2. POPULATION PARAMETERS
# model parameters
modelParameters = list(
  ModelParameter( name = "V",    distribution = LogNormal( mu = 0.2, omega = sqrt(0.25) )),
  ModelParameter( name = "Cl",   distribution = LogNormal( mu = 0.05, omega = sqrt(0.25) ) ),
  ModelParameter( name = "E0",   distribution = LogNormal( mu = 1, omega = sqrt(0.09) ) ),
  ModelParameter( name = "C50",  distribution = LogNormal( mu = 1, omega = sqrt(0.09) ) ),
  ModelParameter( name = "Emax", distribution = LogNormal( mu = 4, omega = sqrt(0.09) ) ) )
```

## Define parameters

```
# Example with other distribution and fixed typical value or fixed IIV
modelParameters = list(
  ModelParameter( name = "V",    distribution = LogNormal( mu = 0.2, omega = sqrt(0.25) ),
  ↪  fixedMu = TRUE),
  ModelParameter( name = "Cl",   distribution = Normal( mu = 0.05, omega = sqrt(0.25)),
  ↪  fixedOmega = TRUE )
)
```

# Define error model

```
# 3. ERROR MODELS
errorModelRespPK = Proportional( output = "RespPK", sigmaSlope = 0.3 )
errorModelRespPD = Constant( output = "RespPD", sigmaInter = 0.15 )
modelError = list( errorModelRespPK, errorModelRespPD )
```

```
# Combined error model
errorModelRespPK = Combined1( output = "RespPK", sigmaInter = 0.15, sigmaSlope = 0.3 )
```

# Define design

```
# 4. DESIGN DEFINITION

## administration
administrationRespPK = Administration( outcome = "RespPK", time = c( 0 ), dose = c( 1 ) )

## sampling times
samplingTimesRespPK = SamplingTimes( outcome = "RespPK", samplings = c( 0.167,  6, 12 ) )
samplingTimesRespPD = SamplingTimes( outcome = "RespPD", samplings = c( 0.167, 6, 12, 20 ) )

## arms definition
arm1 = Arm( name = "BrasTest",
            size = 100,
            administrations = list( administrationRespPK ) ,
            samplingTimes   = list( samplingTimesRespPK, samplingTimesRespPD ) )


design1 = Design( name = "design1", arms = list( arm1 ) )
```

## Compute FIM

```
# 5. FISHER INFORMATION MATRIX COMPUTATION
evaluationPopFim = Evaluation( name = "",
                               modelParameters = modelParameters,
                               modelEquations = modelEquations,
                               modelError =  modelError,
                               designs = list( design1 ),
                               outputs = list( "RespPK", "RespPD" ),
                               fimType = 'population' )

evaluationPopFim = run( evaluationPopFim )
```

## Show results - I

```
show(evaluationPopFim)
```

```
*******************************************
 determinant, condition numbers and d-criterion
*******************************************

determinant: 4.297103e+39
D-criterion: 2008.004
Conditional number (fixed effects): 2366.564
Conditional number (variance components): 19.34486
```

## Show results - II

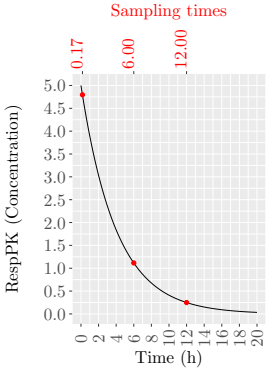
```
*************************************
 Parameters estimation
*************************************
               parametersValues          SE       RSE
_V                         0.20 0.011335255  5.667628
_Cl                        0.05 0.002668098  5.336197
_E0                        1.00 0.034900801  3.490080
_C50                       1.00 0.046621772  4.662177
_Emax                      4.00 0.127584663  3.189617
²_V                        0.25 0.044369220 17.747688
²_Cl                       0.25 0.039175478 15.670191
²_E0                       0.09 0.016987743 18.875270
²_C50                      0.09 0.031318307 34.798119
²_Emax                     0.09 0.013875563 15.417293
_slope_RespPK              0.30 0.019668960  6.556320
_inter_RespPD              0.15 0.011091263  7.394175
```

# Extract results

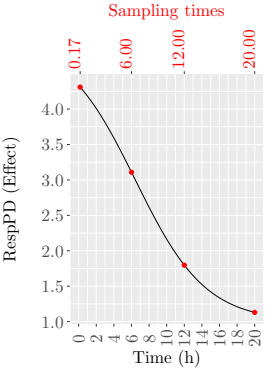
```
> getDcriterion(evaluationPopFim)
[1] 2008.004
> getRSE(evaluationPopFim)
               parametersValues       RSE
_V                         0.20  5.667628
_Cl                        0.05  5.336197
_E0                        1.00  3.490080
_C50                       1.00  4.662177
_Emax                      4.00  3.189617
²_V                        0.25 17.747688
²_Cl                       0.25 15.670191
²_E0                       0.09 18.875270
²_C50                      0.09 34.798119
²_Emax                     0.09 15.417293
_slope_RespPK              0.30  6.556320
_inter_RespPD              0.15  7.394175
```

# Plots

```
plotOptions = list( unitTime = c("h"), unitOutcomes = c("Concentration","Effect")  )
plot_evaluation = plotEvaluation(evaluationPopFim, plotOptions)
```




Design: design1 Arm: BrasTe




Design: design1 Arm: BrasTe

# Reports

```
# Save results & Reports
outputPath = cesuPath
outputFile = "popFIM.html"
Report( evaluationPopFim, outputPath, outputFile, plotOptions )
```

# Optimisation constraints for discrete optimisation

```
# constraints to optimise only PD samplings
administrationConstraintsRespPK = AdministrationConstraints( outcome = "RespPK",
                        doses = list( c( 1 )) )

samplingConstraintsRespPK = SamplingTimeConstraints( outcome = "RespPK",
                        initialSamplings = c( 0.167,  6, 12 ),
                        numberOfsamplingsOptimisable = 3,
                        fixedTimes = c(0.167, 6, 12)) # Fix sampling times

samplingConstraintsRespPD = SamplingTimeConstraints( outcome = "RespPD",
                        initialSamplings = c( 0.167, 1, 2, 6, 8, 10, 12, 16, 20 ),
                        numberOfsamplingsOptimisable = 3 )
```

## Optimisation constraints for continuous optimisation

```
# sampling times constraints
samplingConstraintsSimplexRespPK  = SamplingTimeConstraints( outcome = "RespPK",
                        initialSamplings = c( 0.5, 2, 24, 48, 110 ),
                        samplingsWindows = list( c(1/6,96), c(97,120) ),
                        numberOfTimesByWindows = c(4, 1),
                        minSampling = c(1/6) )
```

# Optimisation arms and design objects

```
arm_opti_1= Arm( name = "BrasTest1",
                 size = 100,
                 administrations = list( administrationRespPK ),
                 samplingTimes   = list( samplingTimesRespPK, samplingTimesRespPD ),
                 administrationsConstraints = list( administrationConstraintsRespPK ),
                 samplingTimesConstraints = list(  samplingConstraintsRespPK,
                 ↪  samplingConstraintsRespPD ) )

design_opti_1 = Design( name = "design1", arms = list( arm_opti_1), numberOfArms = 100 )
```

## Run Optimisation with Multiplicative algorithm

```
optimizationPopFIM = Optimization( name = "PKPD_optimisation_populationFIM",
                                   modelEquations = modelEquations,
                                   modelParameters = modelParameters,
                                   modelError = modelError,
                                   optimizer = "MultiplicativeAlgorithm",
                                   optimizerParameters = list( lambda = 0.99,
                                                               numberOfIterations = 1000,
                                                               weightThreshold = 0.01,
                                                               delta = 1e-04, showProcess = T ),
                                   designs = list( design_opti_1 ),
                                   fimType = "population",
                                   outputs = list( "RespPK", "RespPD" ) )

resOptimizationPopFIM = run( optimizationPopFIM )
```

## Optimisation results

```
====================================
  Optimal design
====================================

  Arms name Number of subjects Outcome Dose Sampling times
1     Arm18                76  RespPK    1  (0.17, 6, 12)
2     Arm18                76  RespPD    .  (0.17, 6, 20)
3     Arm14                12  RespPK    1  (0.17, 6, 12)
4     Arm14                12  RespPD    .   (0.17, 6, 8)
5      Arm7                 9  RespPK    1  (0.17, 6, 12)
6      Arm7                 9  RespPD    .  (0.17, 1, 20)
7     Arm22                 3  RespPK    1  (0.17, 6, 12)
8     Arm22                 3  RespPD    .  (0.17, 8, 20)
```

## Run Optimisation with Simplex algorithm

```
optimizationPopFIM = Optimization( name = "PKPD_optimisation_populationFIM",
                                   modelEquations = modelEquations,
                                   modelParameters = modelParameters,
                                   modelError = modelError,
                                   optimizer = "SimplexAlgorithm",
                                   optimizerParameters = list( pctInitialSimplexBuilding = 20,
                                                               maxIteration = 2,
                                                               tolerance = 1e-6,
                                                               showProcess = TRUE  ),
                                   designs = list( design_opti_1 ),
                                   fimType = "population",
                                   outputs = list( "RespPK", "RespPD" ) )

resOptimizationPopFIM = run( optimizationPopFIM )
```